Technical Report

Exploratory Correlation Analysis and Gaussian Density Evaluation of Academic Assessment Data

Source implementation: `mltak1_merged_colab.py`

A code-centered analysis of data ingestion, exploratory statistics, visualization, and Gaussian density computation

Contents

Abstract

The analyzed implementation is a compact Python/Google Colab workflow that first explores academic assessment data and then evaluates a Gaussian density for a variable named `total`. The program handles file input in both Google Colab and local or Jupyter-style environments, reads an Excel workbook with `pandas`, performs pairwise visualization and correlation analysis on six assessment variables, and then runs a second analysis in which the `total` column is sorted and evaluated with a univariate Gaussian probability density function. The source is organized into two program blocks and contains both exploratory remnants and active code.

The report separates behavior that is directly supported by the source from its theoretical interpretation. In particular, the second program does not transform the stored observations into a new standardized variable. Instead, it computes the value of a normal probability density function at each observed `total` value, using the sample mean and sample standard deviation of the full `total` column. The resulting density values are plotted against the sorted observations. The implementation is best understood as descriptive exploratory data analysis followed by model-based density evaluation, rather than as a complete statistical inference pipeline.

Although the code is small, it demonstrates several useful programming and analytical ideas: environment-aware input handling, spreadsheet ingestion, tabular selection, exploratory visualization, correlation analysis, direct numerical implementation of a probability density function, and sequential data processing. At the same time, the source contains implementation couplings and assumptions that limit robustness, including reliance on a global `data` object inside `gaussian_transform`, the expectation of a `total` column that is not defined in Program 1's explicit column selection, hard-coded plotting limits, and the absence of explicit validation for missing values, zero variance, or non-numeric input.

1 Introduction

The source program addresses a simple data-analysis task: inspect several academic assessment variables for relationships, then examine the distributional shape of an aggregate score. The chosen variables in the first program are `search`, `tak3`, `tak2`, `tak1`, `final`, and `midterm`. In the second program, the analysis shifts to a workbook-wide read and focuses on `total`. The workflow therefore moves from multivariate exploratory analysis to a univariate probabilistic representation.

The implementation works well as a small scientific-computing example because its computational pipeline is visible from input to output. It starts with an explicit Excel filename, attempts a Colab upload, falls back to the local filesystem when the Colab package is unavailable, installs the Excel reader dependency, loads tabular data, generates exploratory visualizations, computes a correlation matrix, sorts an aggregate score, estimates the mean and standard deviation, evaluates the Gaussian density at the observed values, and plots the resulting curve.

This report does not reproduce the source or claim experimental findings that cannot be established from the provided material. The uploaded artifact contains the Python implementation, but not the referenced Excel dataset. Accordingly, the report does not give numerical dataset characteristics, empirical correlations, sample size, or observed density values. Instead, it explains what the code expects from the data, what it computes, how those computations relate to

standard statistical concepts, and what technical assumptions follow from the implementation.

2 Source Scope and Program Structure

The supplied artifact is a Python file whose module-level documentation identifies it as automatically generated from a Colab notebook. The source is 110 lines long and is organized into an input stage, *Program 1*, and *Program 2*. The source header records that the original notebook was stored in Google Colab; the executable content has been flattened into a Python-style script.

The input stage establishes `FILE_NAME = "mltak1Data.xlsx"`. It then attempts to import `google.colab.files` and call `files.upload()`. When the expected filename appears in the uploaded mapping, the original name is retained. If another uploaded file is present, the implementation adopts the first visible uploaded filename. If no file is uploaded, a `FileNotFoundError` is raised. When the Colab module cannot be imported, the source switches to a local/Jupyter path and verifies that the expected filename exists in the current working directory.

After printing the selected input path, the script executes the notebook-style shell command `!pip -q install openpyxl`. This is valid in a Colab/Jupyter code cell but is not valid syntax in an ordinary standalone Python interpreter. This distinction matters because, despite the `.py` extension, the supplied artifact is still intended for a notebook-oriented execution context.

Component	Primary operation	Role in the workflow
Input handling	Colab upload or local file check	Establishes the Excel workbook used by both programs
Program 1	Select six named columns, scatter matrix, correlation heatmap	Exploratory multivariate assessment analysis
Program 2	Read workbook, sort `total`, compute Gaussian density	Univariate distribution-oriented analysis
Visualization	Matplotlib and Seaborn plotting	Converts computed relationships into visual diagnostics

Table 1: Functional decomposition of the supplied implementation.

3 Data and Input Processing

3.1 Input contract

The script assumes the presence of an Excel workbook named `mltak1Data.xlsx`, although it can adopt a different visible filename if the Colab upload contains a file under another name. The workbook therefore functions as the central external dependency of the entire analysis. The source does not create synthetic data and does not contain an embedded dataset.

Program 1 calls

```
data = pd.read_excel(FILE_NAME, usecols=['search','tak3','tak2','tak1','final','
    midterm'])
```

which explicitly restricts the loaded table to six named columns. This is a strong form of input selection because it states the expected schema directly rather than relying on positional indices.

It also implies that those six columns must exist in the workbook for Program 1 to execute successfully.

Program 2 instead calls `pd.read_excel(FILE_NAME, 0)`. The positional argument `0` indicates the first worksheet, and the absence of `usecols` means that the second program reads the workbook's available columns rather than only the six variables used in Program 1. The subsequent statement `data.sort_values('total')` establishes an additional schema requirement: the first sheet must contain a numeric-compatible column named `total`.

3.2 Environment-aware file handling

The upload/fallback mechanism is a practical engineering feature of the source. The code recognizes that a Colab runtime and a local notebook environment expose files differently. The use of a `try/except ImportError` branch separates these environments without duplicating the downstream analysis code.

This design keeps the downstream stages dependent on a single variable, `FILE_NAME`, once input handling is complete. Once the filename is resolved, both programs reuse the same path. It reduces configuration duplication and keeps the data source explicit.

The filename fallback logic has a limitation. If multiple files are uploaded and the expected filename is absent, the code selects the first entry returned by the upload mapping. This is a practical compatibility choice, but it does not verify that the selected workbook is the intended dataset. The source also does not verify workbook contents before proceeding.

4 Technical Foundations

4.1 Exploratory data analysis

Program 1 is exploratory in nature. It does not train a predictive model or perform formal hypothesis testing. Instead, it uses visual and correlation-based summaries to inspect pairwise relationships among the selected assessment variables.

The scatter-matrix operation produces a grid of pairwise plots. Off-diagonal panels represent the relationship between two variables; diagonal panels provide univariate views supplied by pandas' plotting routine. It can help identify linear association, monotonic trends, clusters, scale differences, and potential outliers before a more formal modeling strategy is chosen.

The second visualization is a correlation heatmap constructed from `data.corr()`. With the default correlation method, the resulting matrix is a Pearson-style linear association summary for the numeric variables. For variables X and Y, the Pearson correlation coefficient is

$$r_{XY} = \frac{\operatorname{cov}(X, Y)}{\sigma_X \sigma_Y}, \tag{1}$$

where $\operatorname{cov}(X, Y)$ denotes covariance and σ_X and σ_Y are the corresponding standard deviations. Values near 1 or -1 indicate strong positive or negative linear association, respectively, while values near 0 indicate weak linear association. Correlation does not establish causation and, by itself, does not establish predictive usefulness.

4.2 Gaussian probability density

Program 2 evaluates a univariate normal density. For a scalar variable x, mean μ, and standard deviation $\sigma > 0$, the Gaussian probability density function is

$$f(x \mid \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right). \tag{2}$$

The implementation computes exactly the exponent term and normalization factor appearing in this formula. It uses the sample mean and sample standard deviation of the `total` column as parameter estimates.

A density value is not the probability of an individual continuous observation. Rather, it is a function value whose integral over an interval gives the model-assigned probability mass for that interval. The source instead constructs a smooth parametric reference curve over the observed `total` values; it does not calculate the probability that any particular row is observed.

5 Detailed Analysis of Program 1

5.1 Imports and variable selection

Program 1 imports `pandas`, `numpy`, Matplotlib, the scatter-matrix utility, and Seaborn. The actual scatter matrix is invoked as `pd.plotting.scatter_matrix`, so the imported alias `pdf` is not used for the executed call. Likewise, NumPy is imported, although the only visible NumPy expression in this block is the bare reference `np.linspace`; that statement does not call the function and therefore has no computational effect.

The active data-loading expression requests six named columns. The following commented list constructions are non-executing remnants:

```
#search=list(data['search'])
#tak3=list(data['tak3'])
#tak2=list(data['tak2'])
#tak1=list(data['tak1'])
#final=list(data['final'])
#midterm=list(data['midterm'])
```

Their presence suggests an earlier or exploratory representation in which each column was converted to a Python list. In the current implementation, that conversion is unnecessary because the pandas DataFrame already provides vectorized column operations.

5.2 Scatter-matrix analysis

The statement

```
pff = pd.plotting.scatter_matrix(data.iloc[:, 0:7])
```

requests a scatter matrix using positional column selection. The source DataFrame contains six columns, so the slice from position 0 through 6 is effectively limited by the available column

count and selects the six loaded variables. The upper bound of seven is therefore broader than necessary, but it does not add a seventh feature to this DataFrame.

The returned plotting object is assigned to `pff`. The code then calls `plt.show()` to display the resulting figure. No downstream calculation consumes `pff`; its purpose is purely diagnostic.

This step can reveal whether the assessment variables are approximately linearly related, whether relationships appear nonlinear, whether one assessment has substantially more variability than another, and whether extreme observations warrant further inspection. Because the dataset itself is not supplied, the report cannot state which of those patterns actually occur.

5.3 Correlation heatmap

The next stage creates a Matplotlib figure with a declared size of 10×6 inches and then calls

```
sns.heatmap(data.corr(), center=0, cmap='Greens')
```

The computed correlation matrix is passed directly to Seaborn. The `center=0` setting establishes zero as the conceptual midpoint of the color mapping, which is appropriate for a signed association measure whose direction matters. The use of a single sequential colormap is a visual choice rather than a modification of the correlation values.

There is a minor implementation detail worth noting: `fig, ax = plt.subplots(...)` creates an explicit Matplotlib axes object, but the subsequent Seaborn call does not pass `ax=ax`. Seaborn therefore manages the active axes rather than being explicitly tied to the newly created object. The code remains visually oriented and may still render a heatmap as intended, but the explicit `ax` variable is not used to control the call.

6 Detailed Analysis of Program 2

6.1 Workbook reload and sorting

Program 2 re-imports several packages that were already imported earlier, including NumPy, Matplotlib, pandas, and the scatter-matrix helper. It additionally imports `math` and the top-level `matplotlib` module. The repeated imports are harmless in a notebook-style environment, but they reflect the fact that the two program blocks were developed as separate computational units before being merged.

The source then reads the first worksheet without restricting columns:

```
data = pd.read_excel(FILE_NAME, 0)
datas = data.sort_values('total')
```

The variable `datas` is a DataFrame containing the full sheet sorted by `total`. Immediately afterward, the function call uses `datas['total']`, so the function receives a sorted Series rather than the entire DataFrame.

The comment claiming that a series of data is being created in the range 1 to 50 is not supported by the executable code. No range object or synthetic sequence is created. The actual implementation uses the workbook's observed `total` column. This is an important example of

why source-level traceability matters: the comment describes an intention or earlier version that does not match the executed program.

6.2 Gaussian density function

The function is declared as

```
def gaussian_transform(datas):
```

The parameter name suggests that an input series is being processed. However, the function body computes the distribution parameters from the module-level variable `data` rather than from the function parameter. Specifically,

```
u = data["total"].mean()
sig = data['total'].std()
```

The mean is therefore

$$\mu = \frac{1}{N}\sum_{j=1}^{N} x_j, \tag{3}$$

where x_j denotes an observed `total` value and N is the number of observations included in the mean calculation. The standard deviation is obtained through pandas' default sample-standard-deviation behavior, corresponding to a denominator of $N-1$ when the relevant observations are present:

$$s = \sqrt{\frac{1}{N-1}\sum_{j=1}^{N}(x_j - \bar{x})^2}. \tag{4}$$

The implementation stores this value in `sig`.

The function initializes an empty list `fd`, a scalar accumulator `a`, and the mathematical constant π. It then loops over each scalar value i in the supplied series. The exponent component is computed as

$$a_i = \frac{(i-\mu)^2}{2s^2}, \tag{5}$$

followed by

$$e_i = \exp(-a_i). \tag{6}$$

Finally, the normalizing factor and exponential term are multiplied:

$$f_i = \frac{1}{s\sqrt{2\pi}} e_i. \tag{7}$$

The list of density values is returned in the same iteration order as the supplied Series.

6.3 Why sorting matters here

The call

```
y = gaussian_transform(datas["total"])
```

passes the already sorted `total` series. The density function itself is pointwise: sorting does not change $f(x)$ for a given x. Sorting only changes the order in which the density values are stored

and plotted. This makes the final line plot traverse the observations from the smallest `total` to the largest `total`, producing an ordered curve over the empirical support.

6.4 Plotting and axis limits

The final visualization is created with

```
plt.plot(datas["total"], y)
plt.xlim(6.8,18.8)
plt.show()
```

The x-axis therefore contains the sorted observations and the y-axis contains the corresponding Gaussian density values. The explicit x-axis limit is hard-coded. From the source alone, it is impossible to determine whether 6.8 and 18.8 were derived from the dataset, chosen for readability, or inherited from an earlier experiment. Consequently, the report does not treat those limits as statistically meaningful bounds.

7 Mathematical Formulation

The mathematical core of Program 2 is a Gaussian density evaluation with parameters estimated from the sample. Let the observed aggregate scores be $x_1, \ldots, x_N$. The sample mean is

$$\hat{\mu} = \frac{1}{N} \sum_{j=1}^{N} x_j, \tag{8}$$

and the sample standard deviation is

$$\hat{\sigma} = \sqrt{\frac{1}{N-1} \sum_{j=1}^{N} (x_j - \hat{\mu})^2}. \tag{9}$$

For each sorted observation $x_{(i)}$, the program computes

$$\hat{f}(x_{(i)}) = \frac{1}{\hat{\sigma}\sqrt{2\pi}} \exp\left[-\frac{(x_{(i)} - \hat{\mu})^2}{2\hat{\sigma}^2} \right]. \tag{10}$$

Here $x_{(i)}$ denotes the i-th observation after sorting. The notation $\hat{f}$ emphasizes that the density parameters are estimated from the same sample used for evaluation.

This formulation can be viewed at three levels. At the theoretical level, the Gaussian model is a parametric probability density. At the implementation level, the code evaluates the formula directly with scalar arithmetic inside a Python loop. In practical terms, because the parameters are estimated from the sample, the plotted curve is a fitted descriptive density rather than evidence that the underlying population is Gaussian.

No goodness-of-fit statistic, confidence interval, likelihood-ratio test, or formal normality test appears in the source. The implementation therefore stops at density evaluation and visualization.

8 Execution Workflow

The complete execution sequence can be summarized as a deterministic chain of operations once the workbook is available. First, the runtime resolves the Excel filename. Second, the script ensures the Excel-reading dependency is available in the notebook environment. Third, Program 1 loads the six explicitly named assessment variables. Fourth, the program generates a scatter matrix and a correlation heatmap. Fifth, Program 2 reloads the first worksheet, sorts the full table by `total`, extracts the `total` Series, estimates its mean and standard deviation, evaluates a Gaussian density for each value, and plots the paired $(x, f(x))$ sequence.

The two programs are linked through the shared `FILE_NAME` variable, but they do not share a single in-memory DataFrame contract. Program 1 replaces `data` with a six-column DataFrame, while Program 2 replaces the same variable name with the full worksheet. This reassignment is legal Python behavior, yet it introduces statefulness into the script because the meaning of `data` changes between blocks.

9 Design Decisions and Implementation Rationale

Several implementation choices are understandable from the source, even though their original motivation is not documented.

9.1 Spreadsheet-based data source

Using Excel as the input format is practical for academic assessment data because the source data can be maintained separately from the Python code. The analysis script can therefore operate as a reusable processing layer provided the workbook schema remains compatible.

9.2 Two complementary exploratory views

The scatter matrix and correlation heatmap offer complementary information. The scatter matrix preserves pairwise geometry and can reveal patterns not captured by a single coefficient, while the correlation matrix compresses pairwise linear association into a compact numerical summary. Using both reduces the risk of interpreting a correlation coefficient without viewing the corresponding observations.

9.3 Direct Gaussian implementation

The Gaussian calculation is implemented explicitly instead of calling a specialized statistical library. This makes the mathematical structure visible in the source and is pedagogically useful: the normalizing constant, squared deviation, exponential term, and final product can each be inspected directly. The trade-off is that the implementation must handle numerical edge cases itself and uses a Python-level loop rather than a vectorized calculation.

9.4 Notebook-oriented compatibility

The explicit Colab upload path and notebook shell installation command indicate that interactive execution was a design priority. This is consistent with exploratory data analysis, where an analyst may repeatedly upload a workbook and inspect visual output.

10 Implementation Quality and Technical Considerations

10.1 Global-state coupling

The main software-engineering issue in Program 2 is that `gaussian_transform` depends on the global variable `data`. A function that accepts `datas` would ordinarily be expected to derive its required statistics from that argument. Here the statistics are computed from a different object. The current call is consistent because both objects originate from the same workbook, but the function is not self-contained.

A more modular formulation would compute the mean and standard deviation from the supplied series itself. That change would improve testability and reuse, but it would modify the source rather than describe the implementation as it exists. This report therefore treats it as a quality consideration rather than silently rewriting the algorithm.

10.2 Variance and domain assumptions

The Gaussian formula contains division by σ. If every usable `total` value is identical, then the estimated standard deviation is zero and the denominator vanishes. The source contains no explicit guard for this condition. Likewise, the code assumes that the values passed to `math.exp` are valid numeric scalars.

Missing values introduce another consideration. Pandas mean and standard-deviation calculations can exclude missing observations, but the later explicit iteration over the Series can still encounter missing entries. Passing a missing value through the arithmetic operations and `math.exp` can therefore produce invalid results or raise an exception depending on the representation involved. The source does not explicitly clean or validate missing data before the Gaussian loop.

10.3 Computational complexity

Let N be the number of `total` observations and let p be the number of variables used in the exploratory correlation stage. Sorting the `total` values is typically $O(N \log N)$, while a direct single-pass evaluation of the Gaussian expression after parameter estimation is $O(N)$. Computing all pairwise correlations has a cost that grows with both the sample size and the number of variables; for a small fixed p, its practical cost is dominated by the data operations rather than the visualization.

The Gaussian loop stores one density value per observation, so the additional list storage is $O(N)$. The implementation does not show explicit batching, streaming, parallelism, or GPU acceleration. For a small academic dataset, the loop is operationally simple; for very large datasets, vectorized NumPy operations would generally provide better throughput.

10.4 Unused and redundant elements

The source contains several elements that do not contribute to the final computation. These include the imported scatter-matrix alias `pdf`, the top-level `matplotlib` import in Program 2, the bare `np.linspace` expression, the initial value assignment `a=0` before it is immediately overwritten inside the loop, and several commented-out list conversions. There are also repeated imports across the two programs.

These items do not make the workflow incorrect, but they indicate that the source evolved interactively. In an academic software portfolio, they document the exploratory history of the code and also point to opportunities for refactoring if the project is later turned into a reusable package.

11 Reproducibility and Reliability

The implementation includes several features that support reproducibility. It names the expected workbook explicitly, uses deterministic DataFrame sorting, expresses the selected input columns directly, and performs the Gaussian calculation from explicit statistical quantities. There is no random number generation in the visible source, so no random seed is required for the operations shown.

The main dependency for reproduction is the external dataset. The Excel workbook is not included with the analyzed source, so the exact numerical output cannot be reconstructed from the Python file alone. Reproducing the original outputs would additionally require the same workbook contents and a compatible execution environment. The Colab-oriented dependency installation further indicates that the intended environment includes notebook shell support.

The source also does not pin package versions. Because the implementation relies on pandas, NumPy, Matplotlib, Seaborn, openpyxl, and the Google Colab upload interface, version changes could affect warnings, default behavior, or rendering details even though the core mathematical operations are unchanged.

12 Limitations and Technical Considerations

The code supports a concise exploratory workflow, but its scientific interpretation should remain limited. First, the presence of a Gaussian density curve does not establish Gaussianity of the empirical distribution. A fitted normal density can be plotted even when the data are skewed, multimodal, bounded, or otherwise non-normal. A formal distributional assessment would require additional diagnostics that are absent from the source.

Second, the correlation matrix only characterizes pairwise association under its implemented calculation. It does not identify causal mechanisms, control for confounding variables, or replace a predictive or inferential model. Likewise, the scatter matrix is exploratory rather than confirmatory.

Third, the source assumes a particular workbook schema but does not validate it before use. Program 1 requires six named columns, while Program 2 additionally requires a `total` field in the first worksheet. The relationship between these variables is not established in the supplied

code; in particular, the source does not show how `total` was constructed from the assessment components.

Fourth, the hard-coded x-axis range of $[6.8, 18.8]$ may hide observations outside that interval if they exist. Because the dataset is not supplied, the report cannot determine whether the chosen interval truncates any valid observations.

Finally, the program is exploratory rather than production-oriented. It contains no explicit logging layer, unit tests, configuration file, schema validation, exception handling around statistical edge cases, or environment lockfile. Those omissions are not necessarily problems for a small notebook exercise, but they would matter if the implementation were developed into reusable research software.

13 Overall Technical Assessment

The implementation is technically coherent for its intended exploratory workflow. It shows how a spreadsheet input can be connected to statistical inspection and an explicit mathematical computation in one workflow. The first program combines two standard exploratory techniques, while the second program translates the Gaussian density formula directly into executable scalar operations.

From a software-engineering perspective, the most notable strength is the clear separation between environment-aware input setup and the two analysis blocks. The code also preserves a visible mapping between variables in the workbook and variables in the analysis, which is valuable for readability. The main weaknesses are associated with notebook evolution rather than with the central statistical ideas: duplicated imports, residual statements, changing meanings of the global `data` variable, a function whose parameters do not fully determine its calculation, and limited validation of external input.

From a research perspective, the source demonstrates useful foundations in descriptive statistics and computational modeling, but the code should be interpreted as an exploratory analysis rather than a complete statistical study. The implementation does not establish a predictive model, does not evaluate out-of-sample performance, and does not provide inferential or goodness-of-fit evidence. These are not missing pieces that should be invented in the report; they are simply boundaries of what the supplied artifact actually implements.

14 Conclusion

The supplied Python implementation forms a compact two-stage workflow for analyzing academic assessment data. Its first stage reads six named assessment variables and explores their pairwise structure through a scatter matrix and correlation heatmap. Its second stage reads the first worksheet, sorts the `total` variable, estimates its sample mean and sample standard deviation, evaluates a univariate Gaussian density at each observed value, and plots the resulting density values against the sorted scores.

The central mathematical operation is a direct implementation of the normal probability density function, with parameters estimated from the observed sample. This is best characterized as Gaussian density evaluation rather than a transformation of the original observations. The

distinction is important because the original `total` values remain unchanged; only a corresponding density vector is constructed.

Overall, the project shows how spreadsheet data can be connected to exploratory visualization and an explicit statistical model in Python. Its main opportunities for technical improvement concern modularity, input validation, removal of exploratory remnants, and explicit handling of numerical edge cases. Those considerations do not undermine the core analysis, but addressing them would make the implementation more robust and easier to maintain as research software.

Source-to-Report Traceability Notes

The analysis above is based entirely on the supplied source implementation. The input layer is defined in lines 10–41 of the source; Program 1 occupies lines 43–67; and Program 2 occupies lines 69–110. The six-column selection appears in line 51, the scatter matrix and correlation visualization in lines 61–66, workbook-wide loading and sorting in lines 80–81, Gaussian parameter estimation in lines 88–90, the density loop in lines 96–100, and final density evaluation and plotting in lines 106–110. The source also contains a Colab origin note and an upload mechanism, both of which are retained conceptually in this report.

No numerical dataset values, empirical correlations, fitted parameters, or visual conclusions are asserted because the referenced Excel workbook was not part of the supplied artifact. The report therefore describes the computational procedure and its implications without inventing experimental results.